

AKADEMIA GÓRNICZO-HUTNICZA
WYDZIAŁ INŻYNIERMII MECHANICZNEJ I ROBOTYKI
KIERUNEK MECHANICAL ENGINEERING



FEM AND BEM IN ENGINEERING

Flat Model of a Thermal System

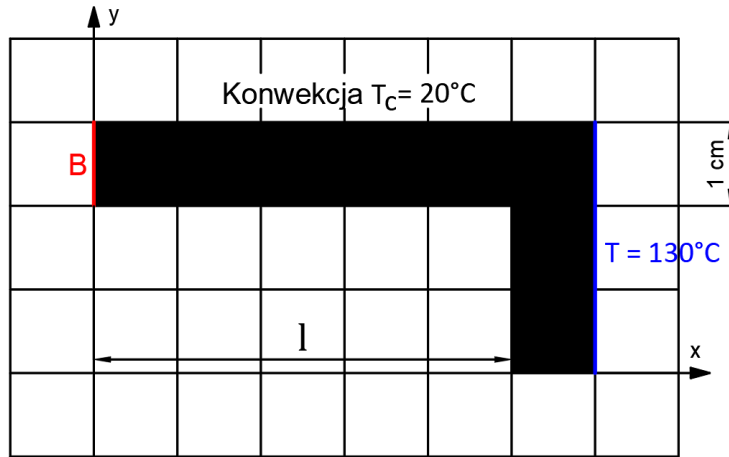
Rafał Sypniewski

Student ID: 410264

1 Problem Analysis

Develop a program in any interpreted or compiled language (e.g., C, Pascal, Matlab, Python, etc.) to solve a problem describing heat flow within a domain of complex geometry. The numerical model should utilize 9-node rectangular elements.

The analyzed problem is presented in Rysunku 1.



Rysunek 1: Analyzed Problem

Scope of work:

- Derive the equations for 9-node elements.
- Determine the steady-state temperature distribution T using the Finite Element Method.
- Perform a mesh independence study.
- Visualize the solution.
- Determine at what length l the temperature at end B does not exceed $40^\circ C$.
- Verify the performed calculations.

2 Matrix Formulation of the Element

2.1 9-Node Element Interpretation

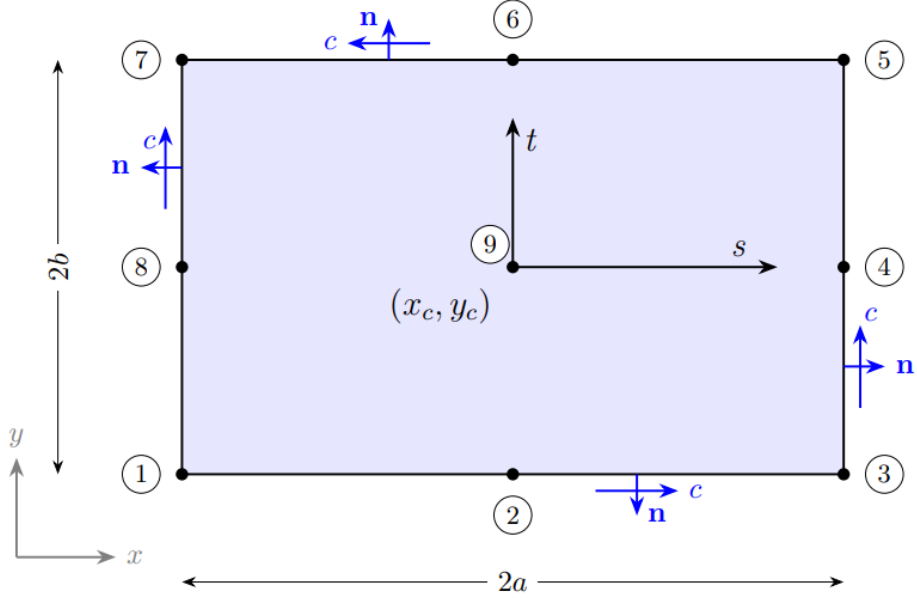
The 9-node element is illustrated in Rysunku 2 along with the local coordinate system (s, t) . The origin of the local coordinate system for the variable is located at the center of each side, and:

$$s = x - x_c$$

$$t = y - y_c$$

$$ds = dx$$

$$dt = dy$$



Rysunek 2: 9-Node Element

The solution inside the element is approximated using a linear combination of basis functions:

$$P(s, t) = \begin{pmatrix} 1 & s & t & st & s^2 & t^2 & s^2 t & st^2 & s^2 t^2 \end{pmatrix}$$

Within area A:

$$u(s, t) = \underbrace{P(s, t) \cdot [A]^{-1}}_{[N]} \cdot \{u_i\}$$

By substituting the coordinates of the 9 nodes (corners, mid-edges, and geometric center) into the polynomial basis, matrix A was obtained:

$$\mathbf{A} = \begin{pmatrix} 1 & -a & -b & ab & a^2 & b^2 & -a^2 b & -ab^2 & a^2 b^2 \\ 1 & 0 & -b & 0 & 0 & b^2 & 0 & 0 & 0 \\ 1 & a & -b & -ab & a^2 & b^2 & -a^2 b & ab^2 & a^2 b^2 \\ 1 & a & 0 & 0 & a^2 & 0 & 0 & 0 & 0 \\ 1 & a & b & ab & a^2 & b^2 & a^2 b & ab^2 & a^2 b^2 \\ 1 & 0 & b & 0 & 0 & b^2 & 0 & 0 & 0 \\ 1 & -a & b & -ab & a^2 & b^2 & a^2 b & -ab^2 & a^2 b^2 \\ 1 & -a & 0 & 0 & a^2 & 0 & 0 & 0 & 0 \\ 1 & 0 & 0 & 0 & 0 & 0 & 0 & 0 & 0 \end{pmatrix}$$

The shape functions were determined from the following relationship:

$$\mathbf{N}(s, t) = P(s, t) \cdot \mathbf{A}^{-1}$$

Determined shape functions:

$$\begin{aligned} N_1 &= \frac{st(a-s)(b-t)}{4a^2b^2} \\ N_2 &= -\frac{t(a^2-s^2)(b-t)}{2a^2b^2} \\ N_3 &= -\frac{st(a+s)(b-t)}{4a^2b^2} \\ N_4 &= \frac{s(a+s)(b^2-t^2)}{2a^2b^2} \\ N_5 &= \frac{st(a+s)(b+t)}{4a^2b^2} \end{aligned}$$

$$\begin{aligned}
N_6 &= \frac{t(b+t)(a^2-s^2)}{2a^2b^2} \\
N_7 &= -\frac{st(b+t)(a-s)}{4a^2b^2} \\
N_8 &= -\frac{s(b^2-t^2)(a-s)}{2a^2b^2} \\
N_9 &= \frac{(a^2-s^2)(b^2-t^2)}{a^2b^2}
\end{aligned}$$

2.2 Gradient Matrix $[B]$ and Conductivity Matrix $[C]$

The first step is to determine the derivatives of the shape functions with respect to the local coordinate system (s, t) , which allows for the construction of the gradient matrix $[B]$:

$$[B] = \begin{bmatrix} \frac{\partial N}{\partial s} \\ \frac{\partial N}{\partial t} \end{bmatrix} = \begin{bmatrix} \frac{\partial N_1}{\partial s} & \frac{\partial N_2}{\partial s} & \cdots & \frac{\partial N_9}{\partial s} \\ \frac{\partial N_1}{\partial t} & \frac{\partial N_2}{\partial t} & \cdots & \frac{\partial N_9}{\partial t} \end{bmatrix}$$

Thermal conductivity matrix $[C]$:

$$[C] = \begin{bmatrix} k_x & 0 \\ 0 & k_y \end{bmatrix}$$

2.3 Element Stiffness Matrix $[k_k]$

The element stiffness matrix responsible for heat conduction is calculated as a double integral over the element area A :

$$[k_k] = \int_{-b}^b \int_{-a}^a [B]^T [D] [B] ds dt$$

2.4 Boundary Conditions of the Third Kind

In the case of convective heat transfer (Robin condition), the boundary parameters α and β are determined by comparing the general formulation with the physical description of convection:

- Parameter $\alpha = -h$, representing energy loss at the boundary.
- Parameter $\beta = h \cdot T_c$, representing thermal forcing from the environment.

The convection contribution to the stiffness matrix $[k_\alpha]$ and the load vector $\{r_\beta\}$ is calculated as an integral along the element boundary C_n :

$$[k_\alpha] = - \int_{C_n} \alpha [N_c]^T [N_c] dc, \quad \{r_\beta\} = \int_{C_n} \beta [N_c]^T dc$$

3 FEM Calculations of Steady-State Temperature Distribution

3.1 Model Parameters

1. Material parameters of the model:

- $kx = ky = 55 \frac{W}{m \cdot K}$
- $h = 85 \frac{W}{m^2 \cdot K}$

2. Geometric parameters:

- width $1cm = 0.01m$
- height of the right rectangle of the model $h = 3cm = 0.03m$
- selected initial length $l = 5cm = 0.05m$

3.2 Boundary Conditions

1. Dirichlet Condition (1st kind) – Constant temperature:
The temperature $T = 130^{\circ}\text{C}$ was applied to the entire edge marked in blue according to Rysunku 1.
2. Robin Condition (3rd kind) – Convection
This condition describes heat exchange with the environment (air) at temperature $T_c = 20^{\circ}\text{C}$ and a heat transfer coefficient $h = 85$. It was applied to the remaining external surfaces of the model.

3.3 Mesh Independence Study

The mesh independence test was conducted for a representative length $l = 0.05\text{ m}$. The mesh refinement process continued until the relative error between iterations fell below the assumed threshold:

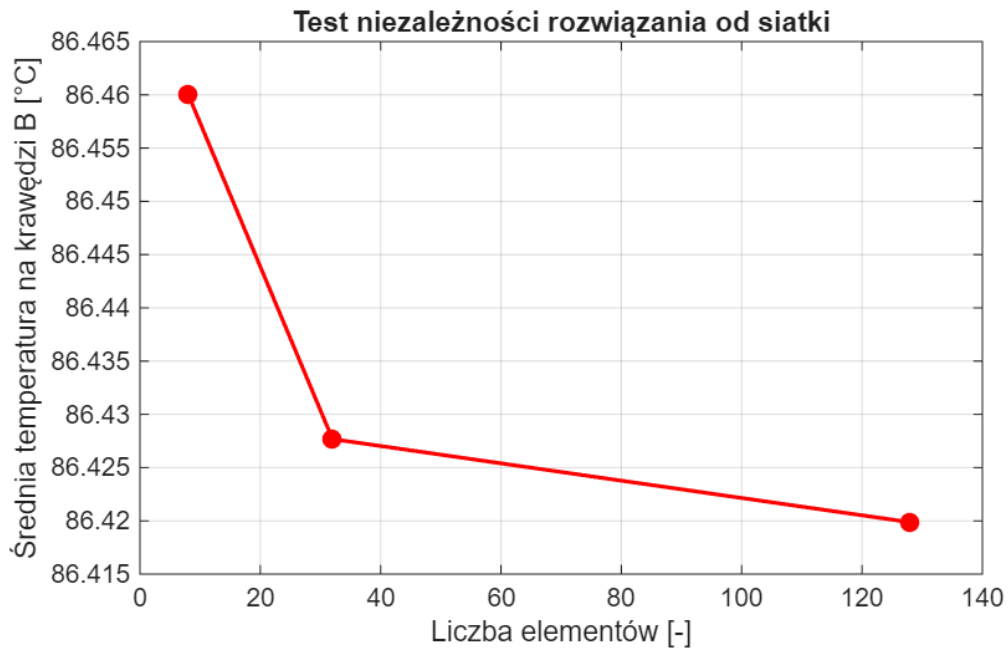
$$\delta = 0.01\%$$

The results obtained in successive mesh refinement steps are presented in Tabeli 1.

Tabela 1: Study of solution convergence relative to the number of elements

Iteration	Elements	Element Size [m]	Temp. B [$^{\circ}\text{C}$]	Abs. Error [$^{\circ}\text{C}$]	Rel. Error [%]
1	8	0.01	86.46	0	0
2	32	0.005	86.428	0.032388	0.037474
3	128	0.0025	86.42	0.0078152	0.0090432

The visualization of the temperature convergence process as a function of the number of elements is shown in Rysunku 3.



Rysunek 3: Solution convergence plot (average temperature at edge B)

Analyzing the results compiled in Tabeli 1, it can be observed that the criterion $\delta = 0.01\%$ was achieved in the 3rd iteration (128 elements). However, even at a size of 0.005 m (32 elements), the relative error is only 0.037% .

Considering numerical stability and the negligible difference in the final result with a significantly lower computational load, an element size of 0.005 m was adopted for further analysis. This choice ensures high accuracy while maintaining short computation time.

3.4 Analysis of Model Length L Influence

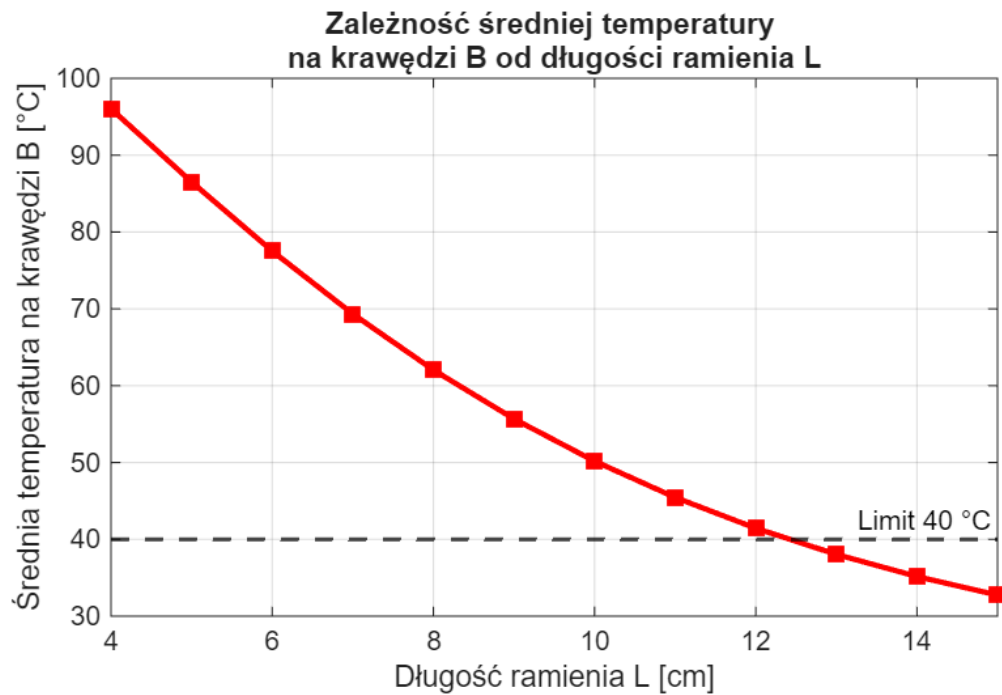
The objective of this stage was to determine the optimal arm length L for which the average temperature at edge B does not exceed 40°C . The iterative process involved increasing the length L from 4 cm (0.04 m)

until a temperature of approximately 35°C was reached, allowing for precise capture of the moment the assumed limit was crossed. Calculations were performed with a constant mesh density using an element size of 0.005 m.

The obtained temperature values are compiled in Tabeli 2, and the trend is shown in Rysunku 4.

Tabela 2: Influence of arm length L on the temperature at control point B

L [cm]	Temp. B [$^{\circ}\text{C}$]
4	96.13
5	86.59
6	77.59
7	69.39
8	62.10
9	55.72
10	50.21
11	45.49
12	41.47
13	38.06
14	35.18
15	32.76



Rysunek 4: Average temperature at edge B vs. arm length L

The analysis shows a clear non-linearity in the temperature drop as the arm length increases. This results from the fact that as the distance from the heat source increases, the surface temperature of the model decreases, which directly reduces the temperature difference relative to the environment and limits the efficiency of convective heat dissipation according to Newton's law. According to the data in Tabeli 2, a length of $L = 12\text{cm}$ yields a temperature of 41.7°C , which still does not meet the design requirements. The condition $T_B \leq 40^{\circ}\text{C}$ is met only after extending the arm to $L = 13\text{cm}$, where the temperature drops to 38.06°C .

4 Final Analysis

The results are presented for the analyzed model using parameters adopted based on previous tests and analyses.

4.1 Model Parameters

1. Material parameters:

- $kx = ky = 55 \frac{W}{m \cdot K}$
- $h = 85 \frac{W}{m^2 \cdot K}$

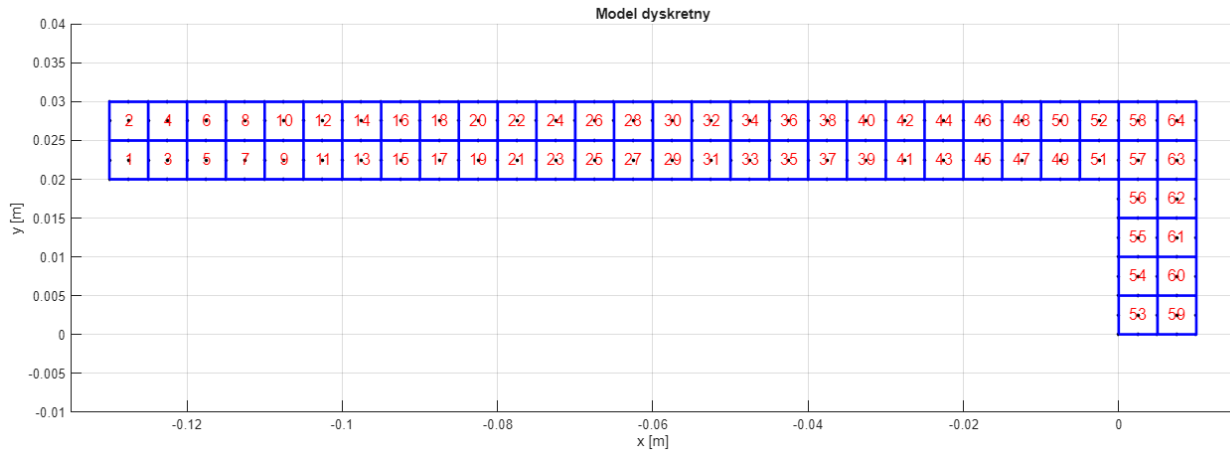
2. Geometric parameters:

- width $1cm = 0.01m$
- height of the right rectangle $h = 3cm = 0.03m$
- selected final length $l = 13cm = 0.13m$

Boundary conditions are consistent with Subsection 3.2.

4.2 Discrete Model

The final numerical model is shown in Rysunku 5. The model consists of 64 elements and 325 nodes.



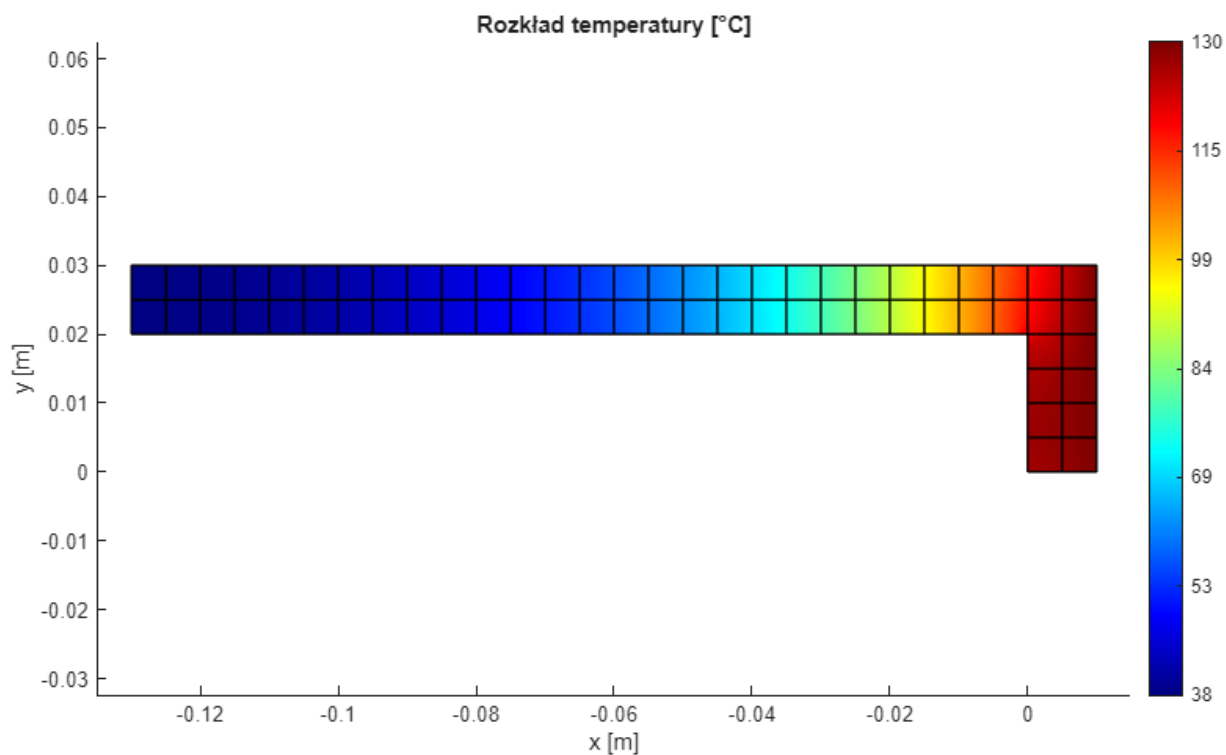
Rysunek 5: Discrete Model

4.3 Analysis Results and Interpretation

The simulation determined the temperature field distribution in the analyzed element. The extreme values and the average temperature on the free edge B are as follows:

- Maximum temperature: $T_{max} = 130,00^{\circ}C$
- Minimum temperature: $T_{min} = 38,03^{\circ}C$
- Average temperature at edge B : $T_B = 38,062^{\circ}C$

Graphical presentation of the results is shown in Rysunku 6.



Rysunek 6: Temperature distribution in the model

Analyzing the distribution, a distinct temperature drop is observed as the distance from the heat source increases. The highest temperature (130°C) occurs on the right vertical arm, directly resulting from the applied Dirichlet boundary condition.

Heat flows along the horizontal arm in the negative x-axis direction. The visible temperature gradient is the result of heat loss to the environment via convection on the top, bottom, and side surfaces. Notably:

- The largest temperature drop occurs at the junction of the arms.
- The minimum temperature ($38,03^{\circ}\text{C}$) occurs at the far left end (edge B), which is furthest from the heat source.

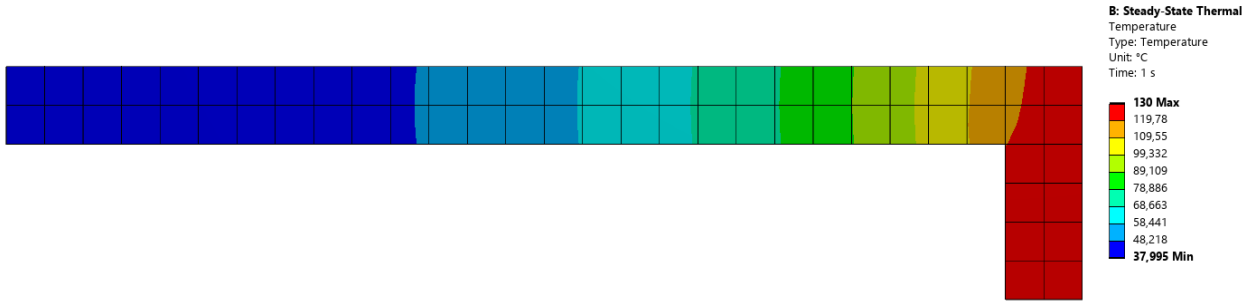
5 Verification of Calculations

To verify the performed calculations and project steps, a flat thermal system model was created for the final assumptions presented in foobar 4. A comparative analysis was conducted in *Ansys* commercial software using the *Steady-State Thermal* module.

All parameters and assumptions were transferred directly, from material properties to boundary conditions. The only difference is the inability to use 9-node elements in the commercial software. However, after reviewing papers by Nam-Sua Lee [1] and Richard H. Macneal [2], both authors demonstrate the superiority of 9-node elements while stating that 8-node elements do not differ significantly and are computationally optimal for professional software. Therefore, the same element size was used for the verification model using 8-node elements.

The distribution for the verification model is shown in Rysunku 7. It is analogous to the one obtained from the Matlab numerical solution shown in Rysunku 6. The average temperature at edge B in the verification model is 38.065°C ; thus, the relative error between the verification solution and the project result is:

$$T_{\delta_B} = \frac{T_{B_{wer}} - T_B}{T_{B_{wer}}} \approx 0.008\%$$



Rysunek 7: Temperature distribution in the verification model

5.1 Conclusion and Summary

Given the relative error of approximately 0.008% between the average temperature at edge *B* in the numerical model and the verification solution from commercial software, the obtained result can be considered correct with high probability.

Programming the solver independently in MATLAB allowed for an understanding of what actually happens "under the hood" of large engineering systems—from manual stiffness matrix definition to boundary condition aggregation. The near-zero difference in results compared to Ansys confirms that the custom implementation of the physics and the choice of 9-node elements were appropriate, and the model correctly represents actual heat flow in a heat sink. The project demonstrated that correct mathematics and attention to mesh detail are key to obtaining reliable results, while also highlighting how demanding it is to solve even a simple geometric problem.

6 Code Used for Calculations

```

1 clear; close all; clc;
2 tic
3 %% Obliczenia symboliczne
4 syms s t a b real
5 syms kx ky h Tc real
6
7 A_mat = [
8     1 -a -b a*b a^2 b^2 -a^2*b -a*b^2 a^2*b^2;
9     1 0 -b 0 0 b^2 0 0 0;
10    1 a -b -a*b a^2 b^2 -a^2*b a*b^2 a^2*b^2;
11    1 a 0 0 a^2 0 0 0 0;
12    1 a b a*b a^2 b^2 a^2*b a*b^2 a^2*b^2;
13    1 0 b 0 0 b^2 0 0 0;
14    1 -a b -a*b a^2 b^2 a^2*b -a*b^2 a^2*b^2;
15    1 -a 0 0 a^2 0 0 0 0;
16    1 0 0 0 0 0 0 0 0
17 ];
18 % (Baza wielomianowa 1, s, t, st, s^2, t^2...)
19 base = [1, s, t, s*t, s^2, t^2, s^2*t, s*t^2, s^2*t^2];
20 N = base * inv(A_mat);
21
22 % Pochodne funkcji kształtu
23 % Macierz gradientu
24 Bs = diff(N, s);
25 Bt = diff(N, t);
26 B = [Bs; Bt];
27 % Macierz przewodności
28 C = [kx 0; 0 ky];
29
30 % Całka sztywności elementu
31 K_sym = int(int(B' * C * B, s, -a, a), t, -b, b);
32
33 % Całki dla konwekcji (Warunki brzegowe)
34 % Wzór: alpha = -h, beta = h*Tc
35 alpha = -h;
36 beta = h * Tc;
37
38 % Dolna krawędź (t = -b)
39 N_dol = subs(N, t, -b);
40 Ka_dol_sym = -int(alpha * (N_dol' * N_dol), s, -a, a);
41 Rb_dol_sym = int(beta * N_dol', s, -a, a);
42
43 % Górna krawędź (t = b)
44 N_gora = subs(N, t, b);
45 Ka_gora_sym = -int(alpha * (N_gora' * N_gora), s, -a, a);
46 Rb_gora_sym = int(beta * N_gora', s, -a, a);
47
48 % Lewa krawędź (s = -a)
49 N_lewa = subs(N, s, -a);
50 Ka_lewa_sym = -int(alpha * (N_lewa' * N_lewa), t, -b, b);
51 Rb_lewa_sym = int(beta * N_lewa', t, -b, b);
52
53 %% Obliczenia numeryczne
54 %DANE LICZBOWE
55 kx_val = 55;
56 ky_val = 55;
57 h_val = 85;
58 Tc_val = 20;
59

```

```

60 % Geometria
61 L_ramie = 0.13;
62 Grubosc = 0.01;
63 H_calosc = 0.03;
64
65 % Parametry dyskretyzacji
66 dx = 0.005;
67 dy = 0.005;
68 a_val = dx/2;
69 b_val = dy/2;
70
71 % parametry
72 K_temp = subs(K_sym, a, a_val);
73 K_temp = subs(K_temp, b, b_val);
74 % parametry materiałowe
75 K_temp = subs(K_temp, kx, kx_val);
76 K_temp = subs(K_temp, ky, ky_val);
77 K_el_num = double(K_temp);
78
79 % Podstawianie do macierzy brzegowych
80 % Dół
81 Ka_temp = subs(Ka_dol_sym, a, a_val); Ka_temp = subs(Ka_temp, h, h_val);
82 Ka_d_num = double(Ka_temp);
83 Rb_temp = subs(Rb_dol_sym, a, a_val); Rb_temp = subs(Rb_temp, h, h_val); Rb_temp =
    ↪ subs(Rb_temp, Tc, Tc_val);
84 Rb_d_num = double(Rb_temp);
85
86 % Góra
87 Ka_temp = subs(Ka_gora_sym, a, a_val); Ka_temp = subs(Ka_temp, h, h_val);
88 Ka_g_num = double(Ka_temp);
89 Rb_temp = subs(Rb_gora_sym, a, a_val); Rb_temp = subs(Rb_temp, h, h_val); Rb_temp =
    ↪ subs(Rb_temp, Tc, Tc_val);
90 Rb_g_num = double(Rb_temp);
91
92 % Lewa
93 Ka_temp = subs(Ka_lewa_sym, b, b_val); Ka_temp = subs(Ka_temp, h, h_val);
94 Ka_l_num = double(Ka_temp);
95 Rb_temp = subs(Rb_lewa_sym, b, b_val); Rb_temp = subs(Rb_temp, h, h_val); Rb_temp =
    ↪ subs(Rb_temp, Tc, Tc_val);
96 Rb_l_num = double(Rb_temp);
97
98 %% Dyskretyzacja
99 % A. Generowanie WĘZŁÓW
100 nodes_x = [];
101 nodes_y = [];
102 licznik_wezlow = 0;
103
104 % Zakresy pętli dla węzłów
105 x_start = -L_ramie;
106 x_end = Grubosc;
107 y_start = 0;
108 y_end = H_calosc;
109
110 step_x_node = dx/2;
111 step_y_node = dy/2;
112
113 % Margines błędu
114 TOL = 1e-6;
115
116 for cx = x_start : step_x_node : (x_end + TOL)
117     for cy = y_start : step_y_node : (y_end + TOL)
118

```

```

119 % Weryfikacja czy znajduje się w zakresie "L"
120 w_pionie = (cx >= -TOL && cx <= Grubosc + TOL) && ...
121         (cy >= -TOL && cy <= H_calosc + TOL);
122
123 w_poziomie = (cx >= -L_ramie - TOL && cx <= TOL) && ...
124         (cy >= H_calosc - Grubosc - TOL && cy <= H_calosc + TOL);
125
126 if w_pionie || w_poziomie
127     licznik_wezlow = licznik_wezlow + 1;
128     nodes_x(licznik_wezlow, 1) = cx;
129     nodes_y(licznik_wezlow, 1) = cy;
130 end
131 end
132 end
133 num_nodes = licznik_wezlow;
134
135 % B. Generowanie elementów
136 ELEMS = [];
137 licznik_elementow = 0;
138
139 for ex = x_start : dx : (x_end - dx + TOL)
140     for ey = y_start : dy : (y_end - dy + TOL)
141
142         % Jeśli środek jest wewnątrz "L", to tworzy element
143         xc = ex + dx/2;
144         yc = ey + dy/2;
145
146         srodek_pion = (xc >= 0 && xc <= Grubosc) && (yc >= 0 && yc <= H_calosc);
147         srodek_poziom = (xc >= -L_ramie && xc <= 0) && (yc >= H_calosc - Grubosc && yc
148             ↪ <= H_calosc);
149
150         if srodek_pion || srodek_poziom
151             % Szukanie ID elementu
152             szukane_x = [ex, ex+dx/2, ex+dx, ex+dx, ex+dx, ex+dx/2, ex, ex,
153                 ↪ ex+dx/2];
154             szukane_y = [ey, ey, ey, ey+dy/2, ey+dy, ey+dy, ey+dy, ey+dy,
155                 ↪ ey+dy/2, ey+dy/2];
156
157             ids_elementu = zeros(1,9);
158             komplet = 1;
159
160             for k = 1:9
161                 id_wezla = 0;
162                 % Przeszukujemy listę wszystkich węzłów
163                 for n = 1:num_nodes
164                     if abs(nodes_x(n) - szukane_x(k)) < TOL && abs(nodes_y(n) -
165                         ↪ szukane_y(k)) < TOL
166                         id_wezla = n;
167                     end
168                 end
169
170                 if id_wezla > 0
171                     ids_elementu(k) = id_wezla;
172                 end
173             end
174
175             if komplet
176                 licznik_elementow = licznik_elementow + 1;
177                 ELEMS(licznik_elementow, :) = ids_elementu;
178             end
179         end
180     end
181 end

```

```

177 end
178 num_elems = size(ELEMS, 1);
179 % wypis parametrów modelu dyskretnego
180 fprintf('Parametrów modelu dyskretnego:\n');
181 fprintf(' Węzłów: %d\n', num_nodes);
182 fprintf(' Elementów: %d\n', num_elems);
183
184 %% Wizualizacja modelu dyskretnego
185 figure(1); hold on; axis equal; grid on;
186 title('Model dyskretny');
187 xlabel('x [m]'); ylabel('y [m]');
188
189 plot(nodes_x, nodes_y, 'k.', 'MarkerSize', 8);
190
191 for e = 1:num_elems
192     wezly = ELEMS(e, :);
193
194
195     x_rys = []; y_rys = [];
196     kolejnosc = [1, 2, 3, 4, 5, 6, 7, 8, 1];
197
198     for k = 1:length(kolejnosc)
199         id = wezly(kolejnosc(k));
200         x_rys(k) = nodes_x(id);
201         y_rys(k) = nodes_y(id);
202     end
203     plot(x_rys, y_rys, 'b-', 'LineWidth', 2);
204     xlim([-0.135 0.015])
205     ylim([-0.010 0.040])
206
207     id_srodek = wezly(9);
208     text(nodes_x(id_srodek), nodes_y(id_srodek), num2str(e), 'Color', 'r', 'FontSize',
        ↪ 12, 'HorizontalAlignment', 'center');
209 end
210
211 %% Agregacja
212 K_global = sparse(num_nodes, num_nodes);
213 F_global = zeros(num_nodes, 1);
214
215 for e = 1:num_elems
216     nodes = ELEMS(e, :);
217
218     % Macierz sztywności
219     K_global(nodes, nodes) = K_global(nodes, nodes) + K_el_num;
220
221     % Pobranie współrzędnych węzłów elementu
222     ex = nodes_x(nodes);
223     ey = nodes_y(nodes);
224
225     % 1. Krawędź DOLNA (y = 0 lub y = H-grubosc pod ramieniem)
226     min_y = min(ey);
227     max_x = max(ex);
228
229     jest_dol_ukladu = 0;
230     if abs(min_y - 0) < TOL
231         jest_dol_ukladu = 1;
232     end
233
234     jest_dol_ramienia = 0;
235     if (abs(min_y - (H_calosc - Grubosc)) < TOL) && (max_x < TOL)
236         jest_dol_ramienia = 1;
237

```

```

238     end
239
240     if jest_dol_ukladu == 1 || jest_dol_ramienia == 1
241         K_global(nodes, nodes) = K_global(nodes, nodes) + Ka_d_num;
242         F_global(nodes) = F_global(nodes) + Rb_d_num;
243     end
244
245     % 2. Krawędź GÓRNA (y = H_calosc)
246     max_y = max(ey);
247     if abs(max_y - H_calosc) < TOL
248         K_global(nodes, nodes) = K_global(nodes, nodes) + Ka_g_num;
249         F_global(nodes) = F_global(nodes) + Rb_g_num;
250     end
251
252     % 3. Krawędź LEWA (x = -L lub x = 0 pod ramieniem)
253     min_x = min(ex);
254
255     jest_lewa_koniec = 0;
256     if abs(min_x - (-L_ramie)) < TOL
257         jest_lewa_koniec = 1;
258     end
259
260     jest_lewa_wewn = 0;
261     if (abs(min_x - 0) < TOL) && (max_y < H_calosc - Grubosc)
262         jest_lewa_wewn = 1;
263     end
264
265     if jest_lewa_koniec == 1 || jest_lewa_wewn == 1
266         K_global(nodes, nodes) = K_global(nodes, nodes) + Ka_l_num;
267         F_global(nodes) = F_global(nodes) + Rb_l_num;
268     end
269 end
270
271
272 %% Warunek Dirichleta (T=130)
273
274 nodes_dirichlet = [];
275 licznik_dir = 0;
276
277 for i = 1:num_nodes
278     if abs(nodes_x(i) - Grubosc) < TOL
279         licznik_dir = licznik_dir + 1;
280         nodes_dirichlet(licznik_dir) = i;
281     end
282 end
283
284 tolerance_dirichlet = 1e15;
285 T_wb = 130;
286
287 for k = 1:length(nodes_dirichlet)
288     id = nodes_dirichlet(k);
289     K_global(id, id) = tolerance_dirichlet;
290     F_global(id) = tolerance_dirichlet * T_wb;
291 end
292
293
294 %% Rozwiązanie i wyniki
295 T = K_global \ F_global;
296
297 disp(['Temperatura Maksymalna: ', num2str(max(T))]);
298 disp(['Temperatura Minimalna: ', num2str(min(T))]);
299

```

```

300 % Obliczanie średniej na końcu B
301 suma_temp_B = 0;
302 licznik_B = 0;
303 for i = 1:num_nodes
304     if abs(nodes_x(i) - (-L_ramie)) < TOL
305         suma_temp_B = suma_temp_B + T(i);
306         licznik_B = licznik_B + 1;
307     end
308 end
309 srednia_B = suma_temp_B / licznik_B;
310 disp(['Srednia temp. na koncu B: ', num2str(srednia_B)]);
311
312
313 %% 8. WYKRES TEMPERATURY (PATCH)
314 figure(2);
315 title('Rozkład temperatury [°C]');
316 xlabel('x [m]'); ylabel('y [m]');
317 axis equal;
318 hold on;
319
320 % Rysowanie po subelementach (tylko wizualizacyjnie)
321 % Każdy element 9-węzłowy dzielimy na 4 mniejsze czworokąty
322 for e = 1:num_elems
323     wezly = ELEMS(e, :);
324
325     ids_1 = wezly([1, 2, 9, 8]);
326     patch(nodes_x(ids_1), nodes_y(ids_1), T(ids_1), 'EdgeColor', 'none', 'FaceColor',
327           ↪ 'interp');
328
329     ids_2 = wezly([2, 3, 4, 9]);
330     patch(nodes_x(ids_2), nodes_y(ids_2), T(ids_2), 'EdgeColor', 'none', 'FaceColor',
331           ↪ 'interp');
332
333     ids_3 = wezly([9, 4, 5, 6]);
334     patch(nodes_x(ids_3), nodes_y(ids_3), T(ids_3), 'EdgeColor', 'none', 'FaceColor',
335           ↪ 'interp');
336
337     ids_4 = wezly([8, 9, 6, 7]);
338     patch(nodes_x(ids_4), nodes_y(ids_4), T(ids_4), 'EdgeColor', 'none', 'FaceColor',
339           ↪ 'interp');
340 end
341
342 cb = colorbar;
343 cb.Ticks = linspace(min(T), max(T), 7);
344 cb.TickLabels = compose('%0f', cb.Ticks);
345 colormap jet;
346 xlim([-L_ramie - 0.005, Grubosc + 0.005]);
347
348 % Obrys
349 for e = 1:num_elems
350     wezly = ELEMS(e, :);
351     kolejnosc = [1, 2, 3, 4, 5, 6, 7, 8, 1];
352     plot(nodes_x(wezly(kolejnosc)), nodes_y(wezly(kolejnosc)), 'k-', 'LineWidth', 0.5);
353 end
354 toc

```

Literatura

- [1] N.-S. Lee and K.-J. Bathe', "Effects of element distortions on the performance of isoparametric elements," *INTERNATIONAL JOURNAL FOR NUMERICAL METHODS IN ENGINEERING*, vol. 36, pp. 3553–3576, 1993.
- [2] R. H. Macneal and R. L. Harder, "Eight nodes or nine?," *International Journal for Numerical Methods in Engineering*, vol. 33, pp. 1049–1058, 1992.